

Introductions to Big Data Systems: Homework 6

Francisco Diego Ariel Romero Contreras

法兰西思科

2025403471

francisco.romero.c@ug.uchile.cl

Word Count

This is a classical map reduce implementation, where we first split by word using the regex given, later map a tuple like $w \rightarrow (w, 1)$, and finally we reduce by key using a sum lambda.

In this particular solution the output is sorted by descendent count and later are taken the first 10 most repeated words.

To run this code the user has to write.

```
python3 word_count.py /hw6_data/pg100.txt
```

The python code is the following.

```
import re
import sys
from pyspark import SparkConf, SparkContext
import time

if __name__ == '__main__':
    conf = SparkConf()
    sc = SparkContext(conf=conf)
    lines = sc.textFile(sys.argv[1])

    first = time.time()

    words = lines.flatMap(lambda line: re.split(r'^\w+', line))
    word_map = words.map(lambda w: (w, 1))
    word_count = word_map.reduceByKey(lambda a, b: a + b)
    word_count = word_count.sortBy(lambda x: -x[1])
    top = word_count.take(10)
    for word, count in top:
        print(f"{word}: {count}")

    last = time.time()

    print("Total program time: %.2f seconds" % (last - first))
    sc.stop()
```

Page Rank

For this problem we have to first transform the plain text to a data structure for links and ranks.

Links RDD can be defined splitting and quitting duplicates as (source, target), and later grouping as (key, targets). To make Ranks RDD we can reuse the Links RDD, making a mapValue over Links with a lambda constant $\frac{1.0}{\text{Number of nodes}}$.

For the page rank calculate we make many iterations of the following calculus.

$$\text{PR}(A) = 1 - d + d \cdot \sum_{I \in V} \frac{\text{PR}(I)}{L(I)}$$

To make this calculus the code uses different parts to achieve this formula.

The function PRL make the $\frac{\text{PR}(I)}{L(I)}$ calculus and return the list for each node:

For page A: $\left[\left(B, \frac{\text{PR}(A,B)}{L(B)} \right), \left(C, \frac{\text{PR}(A,C)}{L(C)} \right) \right]$

L(X) is the amount of outgoing edges for node A. For each node the rank is the same, the important part is that this calculus is made for each node and alter is aggregated.

```
def PRL(pages, rank):  
    n = len(pages)  
    for page in pages:  
        yield (page, rank/n)
```

To calculus the new Ranks we make a join of the links with ranks and for each node we obtain with the function PRL the list of outgoing rank, we flatten this list to later make a reduce by key.

The structure format of the new ranks is:

```
(A, ([out1, out2, ...], rank(A)))  
(B, ([out1, out2, ...], rank(B)))  
...
```

And ends in the following format:

```
(A, [  $\frac{\text{rank}(A)}{n1}$  ,  $\frac{\text{rank}(A)}{n2}$  , ...])  
(B, [  $\frac{\text{rank}(B)}{n3}$  ,  $\frac{\text{rank}(B)}{n4}$  , ...])  
...
```

Later for calculate the ponderated new ranks, we reduce by key summing, calculating this expression $\sum_{I \in V} \frac{\text{PR}(I)}{L(I)}$ And later the damping per node, in this case 0.8 of damping.

```
new_ranks = links.join(ranks)\  
    .flatMap(lambda page: PRL(page[1][0],page[1][1]))\  
    .reduceByKey(lambda x,y: x+y)\  
    .mapValues(lambda rank: 0.2/n + 0.8 * rank)
```

Later we iterate this calculus until achieve convergence or do more that a certain number, now setted as 500. From this results we sort descendent by rank and select the first 5 to print by standard output.

```

import sys
from pyspark import SparkConf, SparkContext
import time

#i in V, [PR(I)/L(I), ...], assuming L = 1 for each leaving edge
def PRL(pages,rank):
    n = len(pages)
    for page in pages:
        yield (page,rank/n)

if __name__ == '__main__':

    # Create Spark context.
    conf = SparkConf()
    sc = SparkContext(conf=conf)
    lines = sc.textFile(sys.argv[1])

    first = time.time()

    links = lines.map(lambda line: tuple(line.split()))\
        .distinct()\
        .groupByKey()\
        .cache()
    n = links.count()
    ranks = links.mapValues(lambda _: 1.0/n)
    ITER = 500
    tol = 1e-6
    for i in range(ITER):
        #[key, [[url1, url2], rank]]
        new_ranks =links.join(ranks)\
            .flatMap(lambda page: PRL(page[1][0],page[1][1]))\
            .reduceByKey(lambda x,y: x+y)\
            .mapValues(lambda rank: 0.2/n + 0.8 * rank)

        diff = ranks.join(new_ranks).map(lambda x: abs(x[1][0] - x[1][1])).sum()
        if diff < tol:
            print(f"Converged after {i+1} iterations (diff = {diff:.8e})")
            ranks = new_ranks
            break
        ranks = new_ranks

    highest = ranks.sortBy(lambda x: -x[1]).take(5)
    print("5 highest:", highest)

    last = time.time()

    print("Total program time: %.2f seconds" % (last - first))
    sc.stop()

```